Reporte técnico de elaboración de la iteración #3 del sistema transaccional EPSAndes

Juan C. Castiblanco Rodriguez, Camila Pantoja Gómez
Contexto de Presentación del documento
Universidad de los Andes, Bogotá, Colombia
{jc.castiblancor, c.pantoja}@uniandes.edu.co
Fecha de presentación: diciembre 4 del 2019

Tabla de contenido

Tabla de contenido 1
1 Introducción 1
2 Análisis 2
2.1 Ajustes en el modelo conceptual 2
1.1 Ajustes en el modelo relacional 2
1.1.1 PrestacionServicio 2
1.1.2 Campania 2
3 Diseño físico 3
3.1 Índices preexistentes 3
3.2 Índices utilizados 4
3.3 Análisis de requerimientos funcionales de consulta 4
3.3.1 RFC1 4
3.3.2 RFC2 5
3.3.3 RFC3 6
3.3.4 RFC4 8
3.3.5 RFC5 9
3.3.6 RFC6 10
3.3.7 RFC7 11
3.3.8 RFC8 12
3.3.9 RFC9 14
3.3.10 RFC10 16
3.3.11 RFC11 20
3.3.12 RFC12 24
4 Construcción de la aplicación 25
4.1 Diseño de datos 25
4.2 Análisis de optimización 25
5 Resultados 26
5.1 Resultados logrados 26
5.2 Resultados no logrados 26

1 Introducción

En este documento se presenta el resumen de la elaboración de la tercera iteración del proyecto del curso de Sistemas Transaccionales basado en el caso de estudio de una aplicación transaccional para una EPS: EPSAndes.

2 Análisis

2.1 Ajustes en el modelo conceptual

Solo se añadieron dos asociaciones para poder asociar las prestaciones de servicio con el organizador de campaña en la aplicación y así cumplir con la privacidad de la información. Las asociaciones introducidas son:

- `campania: Campania` como relación de `PrestacionServicio`
- `organizador: OrganizadorCampania` como relación de `Campania`

A continuación se halla la representación del negocio en UML que permite el funcionamiento del negocio.

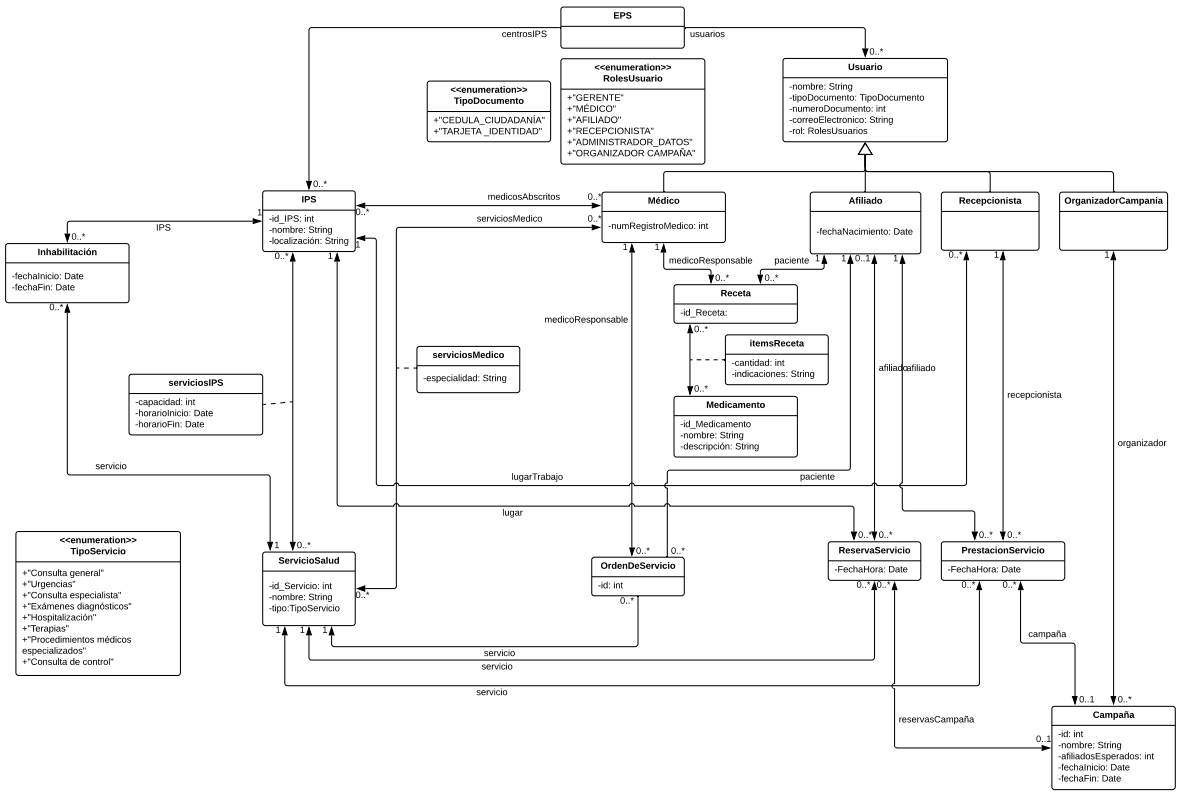


Figura 1. Modelo conceptual de EPSAndes

1.1 Ajustes en el modelo relacional

Las modificaciones correspondientes en el esquema de tablas se dan a continuación.

1.1.1 PrestacionServicio

numDoc	id_Servicio	id_IPS	fechaHora	id_Recepcionista	campania
Number	Number	Number	Date	Number	Number
FK$_{\text{Afiliados.numDoc}}$, NN	PK, FK$_{\text{Servicios.id_Servicio}}$	PK, FK$_{\text{IPS.id_IPS}}$	PK, UA	FK$_{\text{Recepcionistas.numDoc}}$	FK$_{\text{Campania.id}}$

1.1.2 Campania

id	nombre	afiliadosEsperados	fechaInicio	fechaFin	organizador
Number	Number	Number	Date	Date	Number
PK	NN	CK(>0)	NN	NN	FK$_{\text{Usuarios.id}}$

3 Diseño físico

3.1 Índices preexistentes

En la

Figura 2 se muestran los índices creados automáticamente por Oracle. Se puede ver que estos corresponden a las restricciones de llave primaria y de no duplicados. Los de llave primaria fueron creados porque de todas formas se guarda la información en un árbol B+ integrado. Los de *uniqueness constraints* se crean por eficiencia, asumiendo que se pueden utilizar, por ejemplo, como foreign keys.

No obstante, los índices nos ayudan a buscar, filtrar o unir tablas. En ninguno de estos casos nos sirven los índices asociados a restricciones de no duplicados puesto que estas restricciones están puestas sobre columnas que nunca son criterio de búsqueda, filtrado o llaves foráneas.

En cuanto a los índices creados por llaves primarias, estos sí son de gran tanto en la modificación de las tablas como en las consultas. En la inserción, actualización y borrado de registros estos índices permiten mayor eficiencia ya que el orden de estos corresponde al orden en el que se guarda la información en memoria.

Por otra parte, en cuanto a las consultas existe una gran dependencia, claramente, con los criterios de filtrado, agrupamiento u ordenamiento de las mismas. En el caso de las consultas requeridas para EPSAndes y por el modelo de datos propuesto se deben hacer muchos *joins* entre distintas tablas por criterio de llaves foráneas que son siempre llaves primarias de otras tablas. Por lo tanto, aquí vienen a ser cruciales los índices primarios basados en las restricciones de llaves primarias. En cuanto a el filtrado de información este se suele hacer sobre las tablas de reservas y prestaciones. En estas tablas, donde se tienen llaves compuestas los índices toman el orden de las columnas para el criterio de ordenamiento. Por lo tanto, ambas están ordenadas primero por servicio, luego por IPS y finalmente por fecha y hora. Es decir que un *join* con la tabla de servicios puede aprovechar estos índices.

	TABLA	NOMBRE	TIPO	COLUMNA
1	AFILIADOS	AFILIADOS_PK	NORMAL	NUMDOC
2	CAMPANIA	CAMPANIA_PK	NORMAL	ID
3	INHABILITACION	INHABILITACION_PK	NORMAL	SERVICIO
4	INHABILITACION	INHABILITACION_PK	NORMAL	IPS
5	INHABILITACION	INHABILITACION_PK	NORMAL	FECHAINICIO
6	IPS	IPS_PK	NORMAL	ID_IPS
7	IPS	UN_IPS_LOCALIZACION	NORMAL	LOCALIZACION
8	ITEMSRECETA	ITEMSRECETA_PK	NORMAL	ID_RECETA
9	ITEMSRECETA	ITEMSRECETA_PK	NORMAL	ID_MEDICAMENTO
10	MEDICAMENTOS	MEDICAMENTOS_PK	NORMAL	ID_MEDICAMENTO
11	MEDICOS	MEDICOS_PK	NORMAL	NUMDOC
12	MEDICOSABSCRITOS	MEDICOSABSCRITOS_PK	NORMAL	NUMDOC
13	MEDICOSABSCRITOS	MEDICOSABSCRITOS_PK	NORMAL	ID_IPS
14	ORDENES	ORDENES_PK	NORMAL	ID
15	PRESTACIONSERVICIO	PRESTACIONSERVICIO_PK	NORMAL	ID_SERVICIO
16	PRESTACIONSERVICIO	PRESTACIONSERVICIO_PK	NORMAL	ID_IPS
17	PRESTACIONSERVICIO	PRESTACIONSERVICIO_PK	NORMAL	FECHAHORA
18	RECEPCIONISTAS	RECEPCIONISTAS_PK	NORMAL	NUMDOC
19	RECETAS	RECETA_PK	NORMAL	ID_RECETA
20	RESERVASERVICIO	RESERVASERVICIO_PK	NORMAL	ID_SERVICIO
21	RESERVASERVICIO	RESERVASERVICIO_PK	NORMAL	ID_IPS
22	RESERVASERVICIO	RESERVASERVICIO_PK	NORMAL	FECHAHORA
23	ROLUSUARIO	ROLUSUARIO_PK	NORMAL	ID_ROL
24	ROLUSUARIO	UN_ROLUSUARIO_NOMBRE	NORMAL	NOMBRE
25	SERVICIOS	SERVICIOS_PK	NORMAL	ID_SERVICIO
26	SERVICIOSIPS	SERVICIOSIPS_PK	NORMAL	ID_IPS
27	SERVICIOSIPS	SERVICIOSIPS_PK	NORMAL	ID_SERVICIO
28	SERVICIOSMEDICOS	SERVICIOSMEDICOS_PK	NORMAL	NUMDOC
29	SERVICIOSMEDICOS	SERVICIOSMEDICOS_PK	NORMAL	ID_SERVICIO
30	TIPOSERVICIO	TIPOSERVICIO_PK	NORMAL	ID
31	USUARIOS	USUARIOS_PK	NORMAL	NUMDOC

Figura 2: Resultado de consulta de índices

3.2 Índices utilizados

Dado que la mayoría de consultas están basadas en la prestación de servicios teniendo en cuenta tanto servicios como fecha e IPS, estos son los principales atributos a considerar a la hora de crear índices para optimizar las consultas. En cuanto a servicios, ya el arbol B+ que se crea automáticamente por la llave primaria es aprovechable pues esta es la primera columna de la llave compuesta; este se utiliza para hacer todos los joins de esta tabla con la tabla Servicios y adicionalmente para filtrar por servicio en RF9, RFC2, RFC3, RFC5, RFC6, RFC7, RFC8, RFC9, RFC10, . De esta manera, una buena idea para índice y que definitivamente es el criterio más utilizado es el de fecha de prestación de servicio, pues también es usado en RFC1, RFC2, RFC5, RFC6, RFC7, RFC8, . Por lo tanto se introduce el siguiente índice:

- Tabla: PrestacionServicio
- Columna (s): FechaHora
- Tipo árbolB+: es el que se utiliza para búsquedas por rango, que es lo que se requiere en este caso.
- Secundario: no es *clustered* por el hecho de que la tabla en memoria no está ordenada por criterio de fechaHora.
- Mantenimiento y almacenamiento: El costo de almacenamiento y de mantenimiento es compensado por el gran uso que se le da a este índice en todas las consultas. Esto especialmente por ser un índice necesariamente denso, por lo que requiere de un apuntador por cada registro en las hojas. Además, debe ser actualizado cada vez que se presta un servicio, lo cual es en todo momento. No obstante, es más fácil de mantener que un hash table.

Finalmente, se resume del análisis que no vale la pena poner otro índice pues con este quedan cubiertas casi todas las consultas. Si la empresa lo considerara, se podría implementar un índice secundario sobre la columna de IPS en la tabla de prestación de servicios pero esto puede afectar el uso de la aplicación pues es la tabla a la que más se le insertan tuplas ya que se prestan servicios a diario, en cada minuto. No es conveniente que quede con 3 índices para actualizar en cada inserción.

	TABLA	NOMBRE	TIPO	COLUMNA
18	PRESTACIONSERVICIO	RNF_PRESTACIONSERVICIO_FECHA	NORMAL	FECHAHORA

Figura 3: Consulta donde se muestra el índice creado

3.3 Análisis de requerimientos funcionales de consulta

3.3.1 RFC1

Sentencias utilizadas:

```
SELECT IPS.NOMBRE, COUNT(*)
FROM IPS INNER JOIN prestacionServicio
      ON IPS.ID_IPS = prestacionServicio.ID_IPS
WHERE FECHAHORA >? AND FECHAHORA BETWEEN ? AND ?
GROUP BY IPS.NOMBRE
```

Distribución de datos:

En la consulta número 1 se tiene la unión de las tablas de IPS y prestación servicios, en la que se obtiene el numero de servicios prestados en cada IPS, por esto se muestra la totalidad

de la tabla de IPS. En cuanto a la selectividad presente al realizar la consulta se obtiene que, debido a la población de datos se tiene una distribución uniforme de los servicios prestados, siendo estos 36, se tendría que para cada uno hay una selectividad de aproximadamente 2.78% y como se piden las prestaciones en lo que va del año el número de prestaciones se divide a la mitad, ya que se tienen datos desde el año 2018.

Valores específicos de parámetros:

Ya que como parámetro se tiene la fecha actual, el número de datos que analiza la consulta puede variar. Una consulta realizada en le mes de diciembre debe procesar muchos más datos de la tabla de prestación de servicios que una consulta realizada en el mes de febrero.

Planes de consulta de Oracle:

```
Plan hash value: 2725484225

-----------------------------------------------------------------------------------------------
| Id  | Operation              | Name                  | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT       |                       |    15 |   165 |   332   (3)| 00:00:01 |
|   1 |  HASH GROUP BY         |                       |    15 |   165 |   332   (3)| 00:00:01 |
|*  2 |   FILTER               |                       |       |       |            |          |
|*  3 |    INDEX FAST FULL SCAN| PRESTACIONSERVICIO_PK |  122K|  1315K|   329   (2)| 00:00:01 |
-----------------------------------------------------------------------------------------------
```

Tiempos:

2 meses	SQL Execution Time = 81 ms
6 meses	SQL Execution Time = 85 ms
10 meses	SQL Execution Time = 102 ms

Plan de consulta propuesto:

En el plan de consulta se puede ver que el optimizador decidió recorrer toda la tabla de prestaciones de servicio para llevar a cabo los filtros. Luego utilizó hash para seleccionar la respuesta. Los filtros precisamente eran por fecha por lo que se esperaba que utilizara el índice de fechaHora para así encontrar únicamente las tuplas en los rangos deseados.

3.3.2 RFC2

Sentencias utilizadas:

```
SELECT *
FROM
    (
    SELECT SERVICIOS.NOMBRE, count(*) as cuenta
    FROM SERVICIOS NATURAL INNER JOIN RESERVASERVICIO
    GROUP BY SERVICIOS.NOMBRE
    ORDER BY cuenta DESC
    )
```

```
WHERE ROWNUM <20
```

Distribución de datos:

Los datos asociados con esta consulta no dependen de ningún parámetro y están agrupados por nombre de servicios. Es decir, la respuesta siempre es de tamaño de la tabla de servicios. Esta respuesta cambiará de tamaño solo cuando cambie la cantidad de servicios y su complejidad se verá asociada a la tabla de Reservas y su tamaño pues debe recorrerla toda.

Planes de consulta de Oracle:

```
Plan hash value: 1941944234

-------------------------------------------------------------------------------------------
| Id  | Operation                | Name            | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |                 |    19 |  2698 |   328   (6)| 00:00:01 |
|*  1 |  COUNT STOPKEY           |                 |       |       |            |          |
|   2 |   VIEW                   |                 |    36 |  5112 |   328   (6)| 00:00:01 |
|*  3 |    SORT ORDER BY STOPKEY|                 |    36 |   756 |   328   (6)| 00:00:01 |
|   4 |     HASH GROUP BY        |                 |    36 |   756 |   328   (6)| 00:00:01 |
|*  5 |      HASH JOIN           |                 |  323K|  6643K|   312   (1)| 00:00:01 |
|   6 |       TABLE ACCESS FULL | SERVICIOS       |    36 |   648 |     3   (0)| 00:00:01 |
|   7 |       TABLE ACCESS FULL | RESERVASERVICIO |  323K|   949K|   309   (1)| 00:00:01 |
-------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter(ROWNUM<20)
   3 - filter(ROWNUM<20)
   5 - access("SERVICIOS"."ID_SERVICIO"="RESERVASERVICIO"."ID_SERVICIO")
```

Tiempos:

SQL Execution Time = 347 ms

Plan de consulta propuesto:

Se puede ver que se debe hacer una lectura exhaustiva de ReservaServicio y que el join lo hace por hash. No hay otra mejora que se pueda hacer sobre esta consulta dado que para contar se debe recorrer todo el archivo. Solo si se agrega una condición de rango por fechas se podría optimizar con un índice por fecha pero no se cuenta con esta ayuda ni con esta necesidad.

3.3.3 RFC3

Sentencias utilizadas:

```
SELECT SERVICIOS.NOMBRE, uso/capacidad
FROM SERVICIOS
    INNER JOIN
```

```
(
SELECT PRESTACIONSERVICIO.ID_SERVICIO AS id, COUNT(*) AS uso
FROM PRESTACIONSERVICIO
GROUP BY PRESTACIONSERVICIO.ID_SERVICIO
) aux1 ON SERVICIOS.ID_SERVICIO = aux1.id
INNER JOIN
(
SELECT SERVICIOSIPS.ID_SERVICIO AS id,
         SUM(SERVICIOSIPS.CAPACIDAD) AS capacidad
FROM SERVICIOSIPS
GROUP BY SERVICIOSIPS.ID_SERVICIO
) aux2 ON aux1.id = aux2.id
```

Distribución de datos:

El tamaño de respuesta de esta tabla depende únicamente del tamaño de la tabla servicios. Además debe contar todas las prestaciones de servicio para poder encontrar las estadísticas así que la complejidad dependerá también del tamaño de PrestacionServicio.

Planes de consulta de Oracle:

```
PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------------------------------------------
Plan hash value: 164383326

-----------------------------------------------------------------------------------------------------
| Id  | Operation                    | Name                  | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |                       |    34 |  2380 |   339   (3)| 00:00:01 |
|*  1 |  HASH JOIN                   |                       |    34 |  2380 |   339   (3)| 00:00:01 |
|   2 |   MERGE JOIN                 |                       |    34 |  1496 |   335   (3)| 00:00:01 |
|   3 |    TABLE ACCESS BY INDEX ROWID| SERVICIOS            |    36 |   648 |     2   (0)| 00:00:01 |
|   4 |     INDEX FULL SCAN          | SERVICIOS_PK          |    36 |       |     1   (0)| 00:00:01 |
|*  5 |    SORT JOIN                 |                       |    34 |   884 |   333   (3)| 00:00:01 |
|   6 |     VIEW                     |                       |    34 |   884 |   332   (3)| 00:00:01 |
|   7 |      HASH GROUP BY           |                       |    34 |   102 |   332   (3)| 00:00:01 |
|   8 |       INDEX FAST FULL SCAN   | PRESTACIONSERVICIO_PK |  287K|   843K|   325   (1)| 00:00:01 |
|   9 |   VIEW                       |                       |    36 |   936 |     4  (25)| 00:00:01 |
|  10 |    HASH GROUP BY             |                       |    36 |   216 |     4  (25)| 00:00:01 |
|  11 |     TABLE ACCESS FULL        | SERVICIOSIPS          |    60 |   360 |     3   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------------------------------------------
   1 - access("AUX1"."ID"="AUX2"."ID")
   5 - access("SERVICIOS"."ID_SERVICIO"="AUX1"."ID")
       filter("SERVICIOS"."ID_SERVICIO"="AUX1"."ID")
```

Tiempos:

```
SQL Execution Time = 145 ms
```

Plan de consulta propuesto:

Se puede apreciar que se recorre toda la tabla de prestacion de servicios puesto que se debe contar para sacar estadísticos sobre los índices de uso. Sin embargo, no es tan sencillo pues además debe filtrarlo mediante un hash y mediante un ordenamiento de la misma (ya en teoría están ordenados por estos criterios) se hace un join con la tabla de servicios.

3.3.4 RFC4

Sentencias utilizadas:

```
SELECT SERVICIOS.*
FROM SERVICIOS INNER JOIN PRESTACIONSERVICIO
       ON SERVICIOS.ID_SERVICIO = PRESTACIONSERVICIO.ID_SERVICIO
WHERE PRESTACIONSERVICIO.ID_RECEPCIONISTA = ?;
```

Distribución de datos:

Por la manera en la que se poblaron los datos y según vida real, no debería haber preferencia sobre ningún recepcionista, por lo que se asume que los servicios se distribuyen de manera uniforme con respecto a los recepcionistas que los registran. De esta manera, si hay 45 recepcionsitas y 300mil prestaciones debería haber cerca de 6,500 tuplas para cada recepcionista. Esta es la respuesta esperada sin importar los valores específicos de parámetros.

Planes de consulta de Oracle:

```
PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------------------------
Plan hash value: 3433929501
-----------------------------------------------------------------------------------------
| Id  | Operation          | Name               | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT   |                    |  5546 |   157K|   346   (1)| 00:00:01 |
|*  1 |  HASH JOIN         |                    |  5546 |   157K|   346   (1)| 00:00:01 |
|   2 |   TABLE ACCESS FULL| SERVICIOS          |    36 |   756 |     3   (0)| 00:00:01 |
|*  3 |   TABLE ACCESS FULL| PRESTACIONSERVICIO |  5635 | 45080 |   343   (1)| 00:00:01 |
-----------------------------------------------------------------------------------------
PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access("SERVICIOS"."ID_SERVICIO"="PRESTACIONSERVICIO"."ID_SERVICIO")
   3 - filter("PRESTACIONSERVICIO"."ID_RECEPCIONISTA"=40121)
Note
-----
   - dynamic statistics used: dynamic sampling (level=2)
   - 1 Sql Plan Directive used for this statement
```

Tiempos:

SQL Execution Time = 130 ms

Plan de consulta propuesto:

Se puede ver que se recorren ambas tablas para hacer un hash join puesto que no hay ninguna otra forma de acortar la búsqueda por recepcionista. No hay ningún índice con este criterio por lo que este es el mejor plan.

3.3.5 RFC5

Sentencias utilizadas:

```
SELECT SERVICIOS.NOMBRE, COUNT(*)
FROM SERVICIOS INNER JOIN PRESTACIONSERVICIO
     ON SERVICIOS.ID_SERVICIO = PRESTACIONSERVICIO.ID_SERVICIO
WHERE PRESTACIONSERVICIO.NUMDOC = ?
      AND PRESTACIONSERVICIO.FECHAHORA
            BETWEEN 'XX-YY-ZZ' AND 'XX-YY-ZZ'
GROUP BY SERVICIOS.NOMBRE
;
```

Distribución de datos:

Si se asume una distribución uniforme, la utilización de un servicio por parte de cualquier usuario debería ser muy similar. Además, el tamaño de la respuesta siempre será igual o menor al de los servicios.

Valores específicos de parámetros:

Se hicieron pruebas con distintos usuarios y rangos de fechas. La complejidad no incrementa significativamente con los rangos de fechas aunque sí el número de cuenta y de bytes que se leen tras hacer los filtros.

Planes de consulta de Oracle:

```
PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------------------------------------------
Plan hash value: 2449894760
-----------------------------------------------------------------------------------------------------
| Id  | Operation                      | Name               | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT               |                    |     2 |    68 |   345   (1)| 00:00:01 |
|   1 |  HASH GROUP BY                 |                    |     2 |    68 |   345   (1)| 00:00:01 |
|*  2 |   FILTER                       |                    |       |       |            |          |
|   3 |    NESTED LOOPS                |                    |     2 |    68 |   344   (1)| 00:00:01 |
|   4 |     NESTED LOOPS               |                    |     2 |    68 |   344   (1)| 00:00:01 |
|*  5 |      TABLE ACCESS FULL         | PRESTACIONSERVICIO |     1 |    16 |   343   (1)| 00:00:01 |
|*  6 |      INDEX UNIQUE SCAN         | SERVICIOS_PK       |     1 |       |     0   (0)| 00:00:01 |
|   7 |     TABLE ACCESS BY INDEX ROWID| SERVICIOS          |     3 |    54 |     1   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------
```

```
   2 - filter(TO_DATE('1-mar-2018')>=TO_DATE('1-jan-2018'))
   5 - filter("PRESTACIONSERVICIO"."NUMDOC"=15218 AND
              "PRESTACIONSERVICIO"."FECHAHORA"<='1-mar-2018' AND
              "PRESTACIONSERVICIO"."FECHAHORA">='1-jan-2018')
PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------------------------------------------
   6 - access("SERVICIOS"."ID_SERVICIO"="PRESTACIONSERVICIO"."ID_SERVICIO")
Note
-----
   - dynamic statistics used: dynamic sampling (level=2)
   - this is an adaptive plan
   - 1 Sql Plan Directive used for this statement
```

Tiempos:

2 meses	SQL Execution Time = 55 ms
6 meses	SQL Execution Time = 124 ms
10 meses	SQL Execution Time = 177 ms

Plan de consulta propuesto:

El plan de consulta propuesto consiste en utilizar el índice de servicios y de recorrer la tabla de prestaciones para filtrar y posteriormente se selecciona la respuesta mediante hash.

3.3.6 RFC6

Sentencias utilizadas:

```
SELECT *
FROM
    (
    select TRUNC( reservaServicio.fechaHora, ? ), count(*) as cuenta
    from SERVICIOS inner join reservaServicio
             on servicios.id_servicio = reservaServicio.id_servicio
    where SERVICIOS.NOMBRE = 'Oftalmologia'
    Group by TRUNC( reservaServicio.fechaHora, ? )
    Order by cuenta desc
    )
WHERE ROWNUM <3
;
```

*Donde ‘?’ es reemplazado por DD para día, IW para semana, MM para mes y YYYY para año.

Distribución de datos:

Las respuestas son de selección fija, por lo que el tamaño de respuesta es igual. Sin embargo, la complejidad aumenta al disminuir la frencuencia de la fecha.

Valores específicos de parámetros:

Al incrementar la cantidad del agrupamiento, se incrementa significativamente la complejidad.

Planes de consulta de Oracle:

```
PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------------------------------
Plan hash value: 2124222681
-----------------------------------------------------------------------------------------------
| Id  | Operation               | Name               | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT        |                    |     2 |    38 |    46   (5)| 00:00:01 |
|*  1 |  COUNT STOPKEY          |                    |       |       |            |          |
|   2 |   VIEW                  |                    |  9256 |  171K|    46   (5)| 00:00:01 |
|*  3 |    SORT ORDER BY STOPKEY|                    |  9256 |  262K|    46   (5)| 00:00:01 |
|   4 |     HASH GROUP BY       |                    |  9256 |  262K|    46   (5)| 00:00:01 |
|   5 |      NESTED LOOPS       |                    |  9256 |  262K|    44   (0)| 00:00:01 |
|*  6 |       TABLE ACCESS FULL | SERVICIOS          |     1 |    18 |     3   (0)| 00:00:01 |
|*  7 |       INDEX RANGE SCAN  | RESERVASERVICIO_PK |  9256 |    99K|    41   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter(ROWNUM<3)
   3 - filter(ROWNUM<3)
   6 - filter("SERVICIOS"."NOMBRE"='Oftalmologia')
   7 - access("SERVICIOS"."ID_SERVICIO"="RESERVASERVICIO"."ID_SERVICIO")
```

Tiempos:

Promedio: SQL Execution Time = 75 ms

Plan de consulta propuesto:

Sin importar el parámetro, el plan de consulta propuesto se hace mediante un escaneo por rango de la tabla de reserva de servicios, aprovechando el índice primario. Además, accede a la tabla de servicios y posteriormente filtra según el número de respuestas solicitadas.

3.3.7 RFC7

Sentencias utilizadas:

```
select aux.numdoc, count(*), sum(cuenta) as suma
from
    (
    select prestacionServicio.numdoc, servicios.tipo, count(*) as cuenta
    from prestacionServicio inner join servicios
         on servicios.id_servicio = prestacionservicio.id_servicio
    group by prestacionServicio.numdoc , servicios.tipo
    ) aux
group by aux.numdoc
having count(*)>1 and sum(cuenta)>1;
```

Distribución de datos:

La respuesta depende realmente de la distribución de servicios sobre los afiliados. Como se espera que sea aleatoria, la respuesta es muy grande y poco realista. Además, no depende de ningún parámetro

Planes de consulta de Oracle:

```
PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------------------------------------
Plan hash value: 1744355607

-----------------------------------------------------------------------------------------------------
| Id  | Operation              | Name               | Rows  | Bytes |TempSpc| Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT       |                    |   101 |  1818 |       |  1350   (1)| 00:00:01 |
|*  1 |  FILTER                |                    |       |       |       |            |          |
|   2 |   HASH GROUP BY        |                    |   101 |  1818 |       |  1350   (1)| 00:00:01 |
|   3 |    VIEW                |                    |   156K|  2745K|       |  1350   (1)| 00:00:01 |
|   4 |     HASH GROUP BY      |                    |   156K|  2135K|  6784K|  1350   (1)| 00:00:01 |
|*  5 |      HASH JOIN         |                    |   287K|  3936K|       |   346   (1)| 00:00:01 |
|   6 |       TABLE ACCESS FULL| SERVICIOS          |    36 |   216 |       |     3   (0)| 00:00:01 |
|   7 |       TABLE ACCESS FULL| PRESTACIONSERVICIO |   287K|  2249K|       |   343   (1)| 00:00:01 |
-----------------------------------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter(COUNT(*)>1 AND SUM("CUENTA")>1)
   5 - access("SERVICIOS"."ID_SERVICIO"="PRESTACIONSERVICIO"."ID_SERVICIO")
Note
PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------------------------------------
   - dynamic statistics used: dynamic sampling (level=2)
   - 1 Sql Plan Directive used for this statement
```

Tiempos:

SQL Execution Time = 670 ms

Plan de consulta propuesto:

Se accede a ambas tablas servicios y prestaciones puesto que se debe hacer un conteo. Así pues, luego se hace un hash join y se filtra nuevamente la respuesta. Toma mucho tiempo por la cantidad de tuplas que devuelve.

3.3.8 RFC8

Sentencias utilizadas:

```
select servicios.nombre
from servicios left outer join
    (
    select aux.nombre
```

```
        from
            (
            select servicios.nombre,
                    TRUNC( reservaServicio.fechaHora,'IW' )
            from SERVICIOS inner join reservaServicio
                    on servicios.id_servicio = reservaServicio.id_servicio
            Group by servicios.nombre,
                      TRUNC( reservaServicio.fechaHora, 'IW' )
            having count(*) >2
            ) aux
        group by aux.nombre
        having count(*)=1
        ) aux2
    on servicios.nombre = aux2.nombre
    where aux2.nombre is null;
```

Distribución de datos:

Si se asume una distribución uniforme, la utilización de un servicio por parte de cualquier usuario debería ser muy similar. Además, el tamaño de la respuesta siempre será igual o menor al de los servicios. No depende de parámetros.

Planes de consulta de Oracle:

```
PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------------------------------
Plan hash value: 624521028
-----------------------------------------------------------------------------------------------
| Id  | Operation                 | Name            | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT          |                 |    35 |  5040 |   323   (4)| 00:00:01 |
|*  1 |  HASH JOIN ANTI           |                 |    35 |  5040 |   323   (4)| 00:00:01 |
|   2 |   TABLE ACCESS FULL       | SERVICIOS       |    36 |   540 |     3   (0)| 00:00:01 |
|   3 |   VIEW                    |                 |     1 |   129 |   320   (4)| 00:00:01 |
|*  4 |    FILTER                 |                 |       |       |            |          |
|   5 |     HASH GROUP BY         |                 |     1 |    15 |   320   (4)| 00:00:01 |
|   6 |      VIEW                 |                 | 16198 |   237K|   320   (4)| 00:00:01 |
|*  7 |       FILTER              |                 |       |       |            |          |
|   8 |        HASH GROUP BY      |                 | 16198 |   458K|   320   (4)| 00:00:01 |
|*  9 |         HASH JOIN         |                 |   323K|  9174K|   313   (1)| 00:00:01 |
|  10 |          TABLE ACCESS FULL| SERVICIOS       |    36 |   648 |     3   (0)| 00:00:01 |
|  11 |          TABLE ACCESS FULL| RESERVASERVICIO |   323K|  3479K|   309   (1)| 00:00:01 |
-----------------------------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------
PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------------------------------
   1 - access("SERVICIOS"."NOMBRE"="AUX2"."NOMBRE")
   4 - filter(COUNT(*)=1)
   7 - filter(COUNT(*)>2)
```

```
9 - access("SERVICIOS"."ID_SERVICIO"="RESERVASERVICIO"."ID_SERVICIO")
```

Tiempos:

SQL Execution Time = 355 ms

Plan de consulta propuesto:

Se accede nuevamente a todas las tablas pues se debe hacer un conteo de las reservas. No hay otra opción más que recorrer todo.

3.3.9 RFC9

Sentencias utilizadas:

```
select usuarios.nombre, afiliados.*, servicios.tipo,
       prestacionservicio.id_servicio, prestacionservicio.id_ips,
       prestacionservicio.fechahora
from usuarios, afiliados, prestacionservicio, servicios ,campania
where usuarios.numdoc = afiliados.numdoc
      and afiliados.numdoc = prestacionservicio.numdoc
      and servicios.id_servicio = prestacionservicio.id_servicio
      and prestacionservicio.fechahora
            between '8-feb-18' and '15-feb-18'
      and servicios.id_servicio = 4
      and servicios.tipo = 1
      and prestacionservicio.ID_IPS = 1
      and prestacionservicio.campania = campania.id
      and campania.organizador = 40148
order by prestacionservicio.fechahora
;
```

Distribución de datos:

Esta consulta depende en su tamaño de respuesta de los filtros que se pongan. Por ejemplo, si se ponen rangos de fechas muy pequeños, puede haber menos servicios que se provean y menos usuarios que los utilicen.

Valores específicos de parámetros:

Específicamente se analiza la diferencia entre las prestaciones por parte de la campaña y por parte del gerente pues hay una diferencia significativa ya que muy pocas prestaciones de servicio son de una campaña. Pueden crecer las respuestas hasta en 5 veces dependiendo de los rangos de fechas.

Planes de consulta de Oracle:

```
PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------------------------------------------------
Plan hash value: 4165949644
-----------------------------------------------------------------------------------------------------------------
| Id  | Operation                                | Name                  | Rows  | Bytes | Cost (%CPU)| Time     |
```

```
-----------------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT                          |                       |    66 |  3894 |   163   (1)| 00:00:01 |
|   1 |  SORT ORDER BY                            |                       |    66 |  3894 |   163   (1)| 00:00:01 |
|*  2 |   FILTER                                  |                       |       |       |            |          |
|   3 |    NESTED LOOPS                           |                       |    66 |  3894 |   162   (0)| 00:00:01 |
|   4 |     NESTED LOOPS                          |                       |    66 |  3894 |   162   (0)| 00:00:01 |
|*  5 |      HASH JOIN                            |                       |    66 |  2508 |    96   (0)| 00:00:01 |
|   6 |       NESTED LOOPS                        |                       |    66 |  1650 |    66   (0)| 00:00:01 |
|*  7 |        TABLE ACCESS BY INDEX ROWID        | SERVICIOS             |     1 |     6 |     1   (0)| 00:00:01 |
|*  8 |         INDEX UNIQUE SCAN                 | SERVICIOS_PK          |     1 |       |     0   (0)| 00:00:01 |
|   9 |        TABLE ACCESS BY INDEX ROWID BATCHED| PRESTACIONSERVICIO    |    66 |  1254 |    65   (0)| 00:00:01 |
|* 10 |         INDEX RANGE SCAN                  | PRESTACIONSERVICIO_PK |    63 |       |     2   (0)| 00:00:01 |
|  11 |       TABLE ACCESS FULL                   | AFILIADOS             | 40000 |   507K|    30   (0)| 00:00:01 |
|* 12 |      INDEX UNIQUE SCAN                    | USUARIOS_PK           |     1 |       |     0   (0)| 00:00:01 |
|  13 |     TABLE ACCESS BY INDEX ROWID           | USUARIOS              |     1 |    21 |     1   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------------------------
Predicate Information (identified by operation id):
PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------------------------------------------------
   2 - filter(TO_DATE('15-feb-18')>=TO_DATE('8-feb-18'))
   5 - access("AFILIADOS"."NUMDOC"="PRESTACIONSERVICIO"."NUMDOC")
   7 - filter("SERVICIOS"."TIPO"=1)
   8 - access("SERVICIOS"."ID_SERVICIO"=4)
  10 - access("PRESTACIONSERVICIO"."ID_SERVICIO"=4 AND "PRESTACIONSERVICIO"."ID_IPS"=1 AND
             "PRESTACIONSERVICIO"."FECHAHORA">='8-feb-18' AND "PRESTACIONSERVICIO"."FECHAHORA"<='15-feb-18')
  12 - access("USUARIOS"."NUMDOC"="AFILIADOS"."NUMDOC")

Note
PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------------------------------------------------
   - dynamic statistics used: dynamic sampling (level=2)
   - 1 Sql Plan Directive used for this statement


PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------------------------------------------------
Plan hash value: 1763993158

-----------------------------------------------------------------------------------------------------------------
| Id  | Operation                          | Name                  | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT                   |                       |     1 |    80 |    68   (0)| 00:00:01 |
|*  1 |  FILTER                            |                       |       |       |            |          |
|   2 |   NESTED LOOPS                     |                       |     1 |    80 |    68   (0)| 00:00:01 |
|   3 |    NESTED LOOPS                    |                       |     1 |    80 |    68   (0)| 00:00:01 |
|   4 |     NESTED LOOPS                   |                       |     1 |    59 |    67   (0)| 00:00:01 |
|   5 |      NESTED LOOPS                  |                       |     1 |    46 |    66   (0)| 00:00:01 |
|   6 |       NESTED LOOPS                 |                       |     1 |    38 |    66   (0)| 00:00:01 |
|*  7 |        TABLE ACCESS BY INDEX ROWID| SERVICIOS             |     1 |     6 |     1   (0)| 00:00:01 |
|*  8 |         INDEX UNIQUE SCAN          | SERVICIOS_PK          |     1 |       |     0   (0)| 00:00:01 |
|*  9 |        TABLE ACCESS BY INDEX ROWID| PRESTACIONSERVICIO    |     1 |    32 |    65   (0)| 00:00:01 |
|* 10 |         INDEX RANGE SCAN           | PRESTACIONSERVICIO_PK |    63 |       |     2   (0)| 00:00:01 |
|* 11 |       TABLE ACCESS BY INDEX ROWID | CAMPANIA              |     1 |     8 |     0   (0)| 00:00:01 |
|* 12 |        INDEX UNIQUE SCAN           | CAMPANIA_PK           |     1 |       |     0   (0)| 00:00:01 |
|  13 |      TABLE ACCESS BY INDEX ROWID  | AFILIADOS             |     1 |    13 |     1   (0)| 00:00:01 |
|* 14 |       INDEX UNIQUE SCAN            | AFILIADOS_PK          |     1 |       |     0   (0)| 00:00:01 |
|* 15 |     INDEX UNIQUE SCAN              | USUARIOS_PK           |     1 |       |     0   (0)| 00:00:01 |
|  16 |    TABLE ACCESS BY INDEX ROWID     | USUARIOS              |     1 |    21 |     1   (0)| 00:00:01 |
```

```
PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter(TO_DATE('15-feb-18')>=TO_DATE('8-feb-18'))
   7 - filter("SERVICIOS"."TIPO"=1)
   8 - access("SERVICIOS"."ID_SERVICIO"=4)
   9 - filter("PRESTACIONSERVICIO"."CAMPANIA" IS NOT NULL)
  10 - access("PRESTACIONSERVICIO"."ID_SERVICIO"=4 AND "PRESTACIONSERVICIO"."ID_IPS"=1 AND
             "PRESTACIONSERVICIO"."FECHAHORA">='8-feb-18' AND "PRESTACIONSERVICIO"."FECHAHORA"<='15-feb-18')

PLAN_TABLE_OUTPUT
---------------------------------------------------------------------------------------------------------------
  11 - filter("CAMPANIA"."ORGANIZADOR"=40148)
  12 - access("PRESTACIONSERVICIO"."CAMPANIA"="CAMPANIA"."ID")
  14 - access("AFILIADOS"."NUMDOC"="PRESTACIONSERVICIO"."NUMDOC")
  15 - access("USUARIOS"."NUMDOC"="AFILIADOS"."NUMDOC")

Note
-----
   - dynamic statistics used: dynamic sampling (level=2)
   - 1 Sql Plan Directive used for this statement
```

Tiempos:

Plan de consulta propuesto:

Se puede ver que entre más filtros el plan de consulta varía pues del tamaño de respuesta depende mucho el camino que toma el optimizador de Java.

3.3.10 RFC10

Sentencias utilizadas:

```
--CRITERIO SERVICIOS NO UTILIZADOS
SELECT t1.*
FROM SERVICIOS t1 LEFT OUTER JOIN (
        SELECT *
        FROM PRESTACIONSERVICIO --, campania
        WHERE 1=1
        AND fechahora between '8-feb-18' and '15-feb-18'
        --AND prestacionservicio.ID_IPS = 3
        --and prestacionservicio.campania = campania.id
        --and campania.organizador = 40148
        ) AUX_PRESTACION
    ON t1.ID_SERVICIO = AUX_PRESTACION.ID_SERVICIO
WHERE AUX_PRESTACION.FECHAHORA IS NULL
    --AND t1.tipo = 1
ORDER BY t1.nombre
;
```

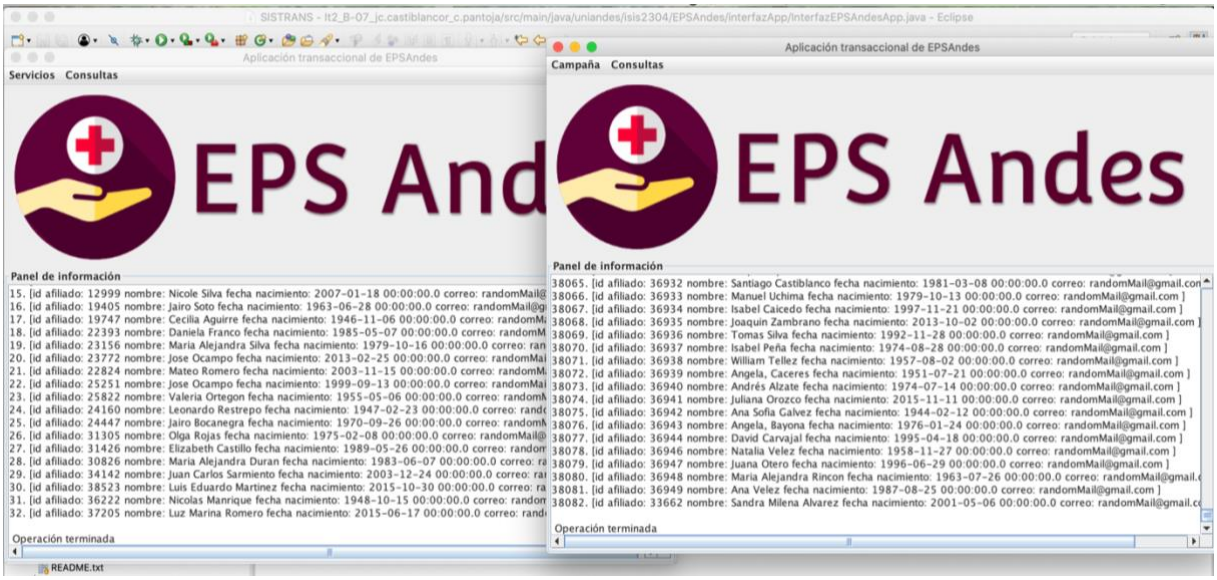
```
--CRITERIO GENTE
SELECT t1.NUMDOC, t1.NOMBRE, t1.FECHANACIMIENTO, t1.CORREO
FROM (
        SELECT USUARIOS.*, AFILIADOS.FECHANACIMIENTO
        FROM AFILIADOS INNER JOIN USUARIOS
             ON AFILIADOS.NUMDOC = USUARIOS.NUMDOC
        ) t1 LEFT OUTER JOIN (
        SELECT PRESTACIONSERVICIO.*
        FROM PRESTACIONSERVICIO, SERVICIOS--,campania
        WHERE PRESTACIONSERVICIO.ID_SERVICIO = SERVICIOS.ID_SERVICIO
        --AND fechahora between '8-feb-18' and '30-mar-18'
        --AND SERVICIOS.TIPO = 3
        --AND SERVICIOS.ID_SERVICIO = 6
        --and prestacionservicio.campania = campania.id
        --and campania.organizador = 40148
        ) AUX_PRESTACION
    ON t1.NUMDOC = AUX_PRESTACION.NUMDOC
WHERE AUX_PRESTACION.FECHAHORA IS NULL
ORDER BY t1.NUMDOC ASC
;

--CRITERIO IPS
SELECT t1.*
FROM (
        SELECT IPS.*, SERVICIOSIPS.ID_SERVICIO,
                 SERVICIOS.NOMBRE NOMSERVICIO, SERVICIOS.TIPO
        FROM IPS, SERVICIOSIPS, SERVICIOS
        WHERE IPS.ID_IPS = SERVICIOSIPS.ID_IPS
             AND SERVICIOSIPS.ID_SERVICIO = SERVICIOS.ID_SERVICIO
     ) t1 LEFT OUTER JOIN (
        SELECT PRESTACIONSERVICIO.*, SERVICIOS.TIPO
        FROM PRESTACIONSERVICIO, SERVICIOS --,campania
        WHERE PRESTACIONSERVICIO.ID_SERVICIO = SERVICIOS.ID_SERVICIO
        --AND fechahora between '27-oct-12' and '29-oct-12'
        --and prestacionservicio.campania = campania.id
        --and campania.organizador = 40148
    ) AUX_PRESTACION
    ON t1.ID_SERVICIO = AUX_PRESTACION.ID_SERVICIO
        AND t1.ID_IPS = AUX_PRESTACION.ID_IPS
WHERE AUX_PRESTACION.FECHAHORA IS NULL
AND t1.ID_SERVICIO = 4
--AND t1.TIPO = 1
ORDER BY t1.ID_IPS desc
;
```

Distribución de datos:

Valores específicos de parámetros:

Donde más cambia es donde la distribución es menos uniforme, es decir si se compara la cantidad de tuplas que salen para un gerente que para un organizador.



Planes de consulta de Oracle:

```
PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------------------------
Plan hash value: 190745914
-----------------------------------------------------------------------------------------------------------
| Id  | Operation                | Name                   | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |                        |  2775 | 88800 |   331   (2)| 00:00:01 |
|   1 |  SORT ORDER BY           |                        |  2775 | 88800 |   331   (2)| 00:00:01 |
|*  2 |   FILTER                 |                        |       |       |            |          |
|*  3 |    HASH JOIN OUTER       |                        |  2775 | 88800 |   330   (1)| 00:00:01 |
|   4 |     TABLE ACCESS FULL    | SERVICIOS              |    36 |   756 |     3   (0)| 00:00:01 |
|*  5 |     INDEX FAST FULL SCAN| PRESTACIONSERVICIO_PK |  2762 | 30382 |   327   (1)| 00:00:01 |

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------
   2 - filter("PRESTACIONSERVICIO"."FECHAHORA" IS NULL)
   3 - access("T1"."ID_SERVICIO"="PRESTACIONSERVICIO"."ID_SERVICIO"(+))
   5 - filter("FECHAHORA"(+)<='15-feb-18' AND "FECHAHORA"(+)>='8-feb-18')
Note
-----
PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------------------------
   - dynamic statistics used: dynamic sampling (level=2)
   - 1 Sql Plan Directive used for this statement


PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------------------------
```

```
Plan hash value: 1035456325
-----------------------------------------------------------------------------------------------------
| Id  | Operation              | Name                | Rows  | Bytes |TempSpc| Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT       |                     |   401 | 27268 |       |   922   (1)| 00:00:01 |
|   1 |  SORT ORDER BY         |                     |   401 | 27268 |       |   922   (1)| 00:00:01 |
|*  2 |   HASH JOIN            |                     |   401 | 27268 |       |   921   (1)| 00:00:01 |
|*  3 |    FILTER              |                     |       |       |       |            |          |
|*  4 |     HASH JOIN OUTER    |                     |   401 | 22055 |  2120K|   891   (1)| 00:00:01 |
|   5 |      TABLE ACCESS FULL| USUARIOS            | 40145 |  1646K|       |   102   (0)| 00:00:01 |
|   6 |      TABLE ACCESS FULL| PRESTACIONSERVICIO  |   287K|  3655K|       |   343   (1)| 00:00:01 |
|   7 |    TABLE ACCESS FULL   | AFILIADOS           | 40000 |   507K|       |    30   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("AFILIADOS"."NUMDOC"="USUARIOS"."NUMDOC")
   3 - filter("PRESTACIONSERVICIO"."FECHAHORA" IS NULL)
   4 - access("USUARIOS"."NUMDOC"="PRESTACIONSERVICIO"."NUMDOC"(+))

PLAN_TABLE_OUTPUT
-------------------------------------------------------------------------------------------------------------------
Plan hash value: 362154909
-----------------------------------------------------------------------------------------------------
| Id  | Operation                      | Name                  | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT               |                       |     2 |   204 |    11   (0)| 00:00:01 |
|*  1 |  FILTER                        |                       |       |       |            |          |
|   2 |   NESTED LOOPS OUTER           |                       |     2 |   204 |    11   (0)| 00:00:01 |
|   3 |    NESTED LOOPS                |                       |     2 |   176 |     3   (0)| 00:00:01 |
|   4 |     NESTED LOOPS               |                       |    15 |  1230 |     3   (0)| 00:00:01 |
|   5 |      TABLE ACCESS BY INDEX ROWID| SERVICIOS             |     1 |    21 |     1   (0)| 00:00:01 |
|*  6 |       INDEX UNIQUE SCAN        | SERVICIOS_PK          |     1 |       |     0   (0)| 00:00:01 |
|   7 |      TABLE ACCESS BY INDEX ROWID| IPS                   |    15 |   600 |     2   (0)| 00:00:01 |
|   8 |       INDEX FULL SCAN DESCENDING| IPS_PK                |    15 |       |     1   (0)| 00:00:01 |
|*  9 |     INDEX UNIQUE SCAN          | SERVICIOSIPS_PK       |     1 |     6 |     0   (0)| 00:00:01 |
|* 10 |    INDEX RANGE SCAN            | PRESTACIONSERVICIO_PK |     1 |    14 |     4   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter("PRESTACIONSERVICIO"."FECHAHORA" IS NULL)

PLAN_TABLE_OUTPUT
-------------------------------------------------------------------------------------------------------------------
   6 - access("SERVICIOS"."ID_SERVICIO"=4)
   9 - access("IPS"."ID_IPS"="SERVICIOSIPS"."ID_IPS" AND "SERVICIOSIPS"."ID_SERVICIO"=4)
  10 - access("PRESTACIONSERVICIO"."ID_SERVICIO"(+)=4 AND
              "IPS"."ID_IPS"="PRESTACIONSERVICIO"."ID_IPS"(+))
```

Tiempos:

```
10:38:14,286 (AWT-EventQueue-0) DEBUG [DataNucleus.Datastore.Native] - SELECT t1.NUMDOC, t1.NOMBRE, t1.FECHANACIMIENTO, t1.CORREO FROM ( SELECT USUARIOS.*, AFILIADOS.FECHANACIMIENTO FROM AFILIADOS INNER JOIN USUARIO
(SELECT PRESTACIONSERVICIO.* FROM PRESTACIONSERVICIO, SERVICIOS, CAMPANIA WHERE PRESTACIONSERVICIO.ID_SERVICIO = SERVICIOS.ID_SERVICIO and prestacionservicio.campania = campania.id and campania.organizador = <40148
AUX_PRESTACION.FECHAHORA IS NULL
10:38:14,365 (AWT-EventQueue-0) DEBUG [DataNucleus.Datastore.Retrieve] - SQL Execution Time = 79 ms
10:38:01,240 (AWT-EventQueue-0) DEBUG [DataNucleus.Datastore.Native] - SELECT t1.NUMDOC, t1.NOMBRE, t1.FECHANACIMIENTO, t1.CORREO FROM ( SELECT USUARIOS.*, AFILIADOS.FECHANACIMIENTO FROM AFILIADOS INNER JOIN USUARIOS ON AFILIADOS.NUMDOC = U
(SELECT PRESTACIONSERVICIO.* FROM PRESTACIONSERVICIO, SERVICIOS WHERE PRESTACIONSERVICIO.ID_SERVICIO = SERVICIOS.ID_SERVICIO ) AUX_PRESTACION ON t1.NUMDOC = AUX_PRESTACION.NUMDOC WHERE AUX_PRESTACION.FECHAHORA IS NULL
10:38:01,467 (AWT-EventQueue-0) DEBUG [DataNucleus.Datastore.Retrieve] - SQL Execution Time = 227 ms
```

Plan de consulta propuesto:

Se puede ver que los planes de ejecución dependen mucho de los criterios que escoge el usuario. Esto porque cambia mucho el tamaño de las respuestas así como los filtros que pongan.

3.3.11 RFC11

Sentencias utilizadas:

```
select t1.week, t1.cantidad, t2.serviciomas, t3.serviciomenos,
        t4.ipsmas, t5.ipsmenos, t6.afiliado, t7.tipomenos, t8.tipomas
from
(   select week, ( select count(*) as cantidad
                   from afiliados)- count(*) as cantidad
    from ( select TRUNC( prestacionServicio.fechaHora, 'IW' ) as week,
                  prestacionservicio.numdoc
           from prestacionServicio
           Group by TRUNC( prestacionServicio.fechaHora, 'IW' ), numdoc
    )
    group by week
) t1 ,
(   select week, serviciomas
    from (
        select TRUNC( prestacionServicio.fechaHora,'IW' ) as week,
            id_servicio as serviciomas, count(*),
            ROW_NUMBER() OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )
            ORDER BY  TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn
        from prestacionservicio
        group by TRUNC( prestacionServicio.fechaHora,'IW' ), id_servicio
        order by TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) desc
   )where rn = 1
) t2,
(   select week, serviciomenos
    from(
        select TRUNC( prestacionServicio.fechaHora,'IW' ) as week,
            id_servicio as serviciomenos, count(*),
            ROW_NUMBER() OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )
            ORDER BY  TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn
        from prestacionservicio
        group by TRUNC( prestacionServicio.fechaHora,'IW' ), id_servicio
        order by TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) asc
   ) where rn = 1
) t3,
(  select week, ipsmas
    From (
        select TRUNC( prestacionServicio.fechaHora,'IW' ) as week,
            id_ips as ipsmas, count(*),
            ROW_NUMBER() OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )
            ORDER BY  TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn
        from prestacionservicio
```

```
        group by TRUNC( prestacionServicio.fechaHora,'IW' ), id_ips
        order by TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) desc
   )where rn = 1
) t4,
(  select week, ipsmenos
   from (
        select TRUNC( prestacionServicio.fechaHora,'IW' ) as week,
            id_ips as ipsmenos, count(*),
            ROW_NUMBER() OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )
            ORDER BY  TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn
        from prestacionservicio
        group by TRUNC( prestacionServicio.fechaHora,'IW' ), id_ips
        order by TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) asc
   )where rn = 1
) t5,
(   select week, afiliado
    from (
        select TRUNC( prestacionServicio.fechaHora,'IW' ) as week,
            numdoc as afiliado, count(*),
            ROW_NUMBER() OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )
            ORDER BY  TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn
        from prestacionservicio
        group by TRUNC( prestacionServicio.fechaHora,'IW' ), numdoc
        order by TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) asc
   )where rn = 1
) t6,
(   select week, tipomenos
    from (
        select TRUNC( prestacionServicio.fechaHora,'IW' ) as week,
            servicios.tipo as tipomenos, count(*),
            ROW_NUMBER() OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )
            ORDER BY  TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn
        from prestacionservicio inner join servicios
        on prestacionservicio.id_servicio = servicios.id_servicio
        group by TRUNC( prestacionServicio.fechaHora,'IW' ), servicios.tipo
        order by TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) desc
    ) where rn = 1
) t7,
(   select week, tipomas
    from (
        select TRUNC( prestacionServicio.fechaHora,'IW' ) as week,
            servicios.tipo as tipomas, count(*),
            ROW_NUMBER() OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )
            ORDER BY  TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn
        from prestacionservicio inner join servicios
        on prestacionservicio.id_servicio = servicios.id_servicio
        group by TRUNC( prestacionServicio.fechaHora,'IW' ), servicios.tipo
        order by TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) asc
   )where rn = 1
) t8
```

```
where t1.week=t2.week and t2.week=t3.week and t3.week=t4.week and t4.week=t5.week
and t5.week=t6.week and t6.week=t7.week and t7.week=t8.week
order by t1.week
;
```

Distribución de datos:

No dependen realmente de los parámetros pues no hay. Pero sí depende de la cantidad de servicios prestados aunque la respuesta siempre tiene el mismo tamaño.

Planes de consulta de Oracle:

```
PLAN_TABLE_OUTPUT
-----------------------------------------------------------------------------------------------------------------------
Plan hash value: 3077715006
-----------------------------------------------------------------------------------------------------------------------
| Id  | Operation                        | Name                  | Rows  | Bytes |TempSpc| Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT                 |                       |     1 |   264 |       |  5395   (2)| 00:00:01 |
|   1 |  SORT AGGREGATE                  |                       |     1 |       |       |            |          |
|   2 |   INDEX FAST FULL SCAN           | AFILIADOS_PK          | 40000 |       |       |    22   (0)| 00:00:01 |
|   3 |  SORT ORDER BY                   |                       |     1 |   264 |       |  5395   (2)| 00:00:01 |
|*  4 |   HASH JOIN                      |                       |     1 |   264 |       |  5394   (2)| 00:00:01 |
|*  5 |    HASH JOIN                     |                       |     1 |   245 |       |  3682   (2)| 00:00:01 |
|*  6 |     HASH JOIN                    |                       |     1 |   210 |       |  3346   (2)| 00:00:01 |
|*  7 |      HASH JOIN                   |                       |     1 |   175 |       |  3010   (2)| 00:00:01 |
|*  8 |       HASH JOIN                  |                       |     1 |   140 |       |  1328   (3)| 00:00:01 |
|*  9 |        HASH JOIN                 |                       |     1 |   105 |       |   996   (3)| 00:00:01 |
|* 10 |         HASH JOIN                |                       |     1 |    70 |       |   664   (3)| 00:00:01 |
|* 11 |          VIEW                    |                       |     1 |    35 |       |   332   (3)| 00:00:01 |
|* 12 |           WINDOW SORT PUSHED RANK|                       |  3594 | 39534 |       |   332   (3)| 00:00:01 |
|  13 |            HASH GROUP BY         |                       |  3594 | 39534 |       |   332   (3)| 00:00:01 |
|  14 |             INDEX FAST FULL SCAN | PRESTACIONSERVICIO_PK |   287K|  3093K|       |   325   (1)| 00:00:01 |
|* 15 |          VIEW                    |                       |     1 |    35 |       |   332   (3)| 00:00:01 |
|* 16 |           WINDOW SORT PUSHED RANK|                       |  3594 | 39534 |       |   332   (3)| 00:00:01 |
|  17 |            HASH GROUP BY         |                       |  3594 | 39534 |       |   332   (3)| 00:00:01 |
|  18 |             INDEX FAST FULL SCAN | PRESTACIONSERVICIO_PK |   287K|  3093K|       |   325   (1)| 00:00:01 |
|* 19 |         VIEW                     |                       |     1 |    35 |       |   332   (3)| 00:00:01 |
|* 20 |          WINDOW SORT PUSHED RANK |                       |  1575 | 17325 |       |   332   (3)| 00:00:01 |
|  21 |           HASH GROUP BY          |                       |  1575 | 17325 |       |   332   (3)| 00:00:01 |
|  22 |            INDEX FAST FULL SCAN  | PRESTACIONSERVICIO_PK |   287K|  3093K|       |   325   (1)| 00:00:01 |
|* 23 |        VIEW                      |                       |     1 |    35 |       |   332   (3)| 00:00:01 |
|* 24 |         WINDOW SORT PUSHED RANK  |                       |  1575 | 17325 |       |   332   (3)| 00:00:01 |
|  25 |          HASH GROUP BY           |                       |  1575 | 17325 |       |   332   (3)| 00:00:01 |
|  26 |           INDEX FAST FULL SCAN   | PRESTACIONSERVICIO_PK |   287K|  3093K|       |   325   (1)| 00:00:01 |
|* 27 |       VIEW                       |                       |     1 |    35 |       |  1682   (1)| 00:00:01 |
|* 28 |        WINDOW SORT PUSHED RANK   |                       |   278K|  3532K|  6784K|  1682   (1)| 00:00:01 |
|  29 |         HASH GROUP BY            |                       |   278K|  3532K|  6784K|  1682   (1)| 00:00:01 |
|  30 |          TABLE ACCESS FULL       | PRESTACIONSERVICIO    |   287K|  3655K|       |   343   (1)| 00:00:01 |
|* 31 |      VIEW                        |                       |     1 |    35 |       |   336   (3)| 00:00:01 |
|* 32 |       WINDOW SORT PUSHED RANK    |                       |   654 | 11118 |       |   336   (3)| 00:00:01 |
|  33 |        HASH GROUP BY             |                       |   654 | 11118 |       |   336   (3)| 00:00:01 |
|* 34 |         HASH JOIN                |                       |   287K|  4780K|       |   329   (1)| 00:00:01 |
|  35 |          TABLE ACCESS FULL       | SERVICIOS             |    36 |   216 |       |     3   (0)| 00:00:01 |
|  36 |          INDEX FAST FULL SCAN    | PRESTACIONSERVICIO_PK |   287K|  3093K|       |   325   (1)| 00:00:01 |
|* 37 |     VIEW                         |                       |     1 |    35 |       |   336   (3)| 00:00:01 |
|* 38 |      WINDOW SORT PUSHED RANK     |                       |   654 | 11118 |       |   336   (3)| 00:00:01 |
|  39 |       HASH GROUP BY              |                       |   654 | 11118 |       |   336   (3)| 00:00:01 |
|* 40 |        HASH JOIN                 |                       |   287K|  4780K|       |   329   (1)| 00:00:01 |
```

```
|  41 |         TABLE ACCESS FULL        | SERVICIOS              |    36 |   216 |        |     3   (0)| 00:00:01 |
|  42 |         INDEX FAST FULL SCAN     | PRESTACIONSERVICIO_PK  |  287K|  3093K|        |   325   (1)| 00:00:01 |
|  43 |    VIEW                          |                        |  278K|  5163K|        |  1711   (1)| 00:00:01 |
|  44 |     HASH GROUP BY                |                        |  278K|   543K|        |  1711   (1)| 00:00:01 |
|  45 |      VIEW                        | VM_NWVW_0              |  278K|   543K|        |  1682   (1)| 00:00:01 |
|  46 |       HASH GROUP BY              |                        |  278K|  3532K|  6784K|  1682   (1)| 00:00:01 |
|  47 |        TABLE ACCESS FULL         | PRESTACIONSERVICIO     |  287K|  3655K|        |   343   (1)| 00:00:01 |
-------------------------------------------------------------------------------------------------------------------------


PLAN_TABLE_OUTPUT
-------------------------------------------------------------------------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------

   4 - access("WEEK"=INTERNAL_FUNCTION("T1"."WEEK"))
   5 - access("WEEK"="WEEK")
   6 - access("WEEK"="WEEK")
   7 - access("WEEK"="WEEK")
   8 - access("WEEK"="WEEK")
   9 - access("WEEK"="WEEK")
  10 - access("WEEK"="WEEK")
  11 - filter("RN"=1)
  12 - filter(ROW_NUMBER() OVER ( PARTITION BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'
              fmiw') ORDER BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'fmiw') DESC )<=1)
  15 - filter("RN"=1)
  16 - filter(ROW_NUMBER() OVER ( PARTITION BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'
              fmiw') ORDER BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'fmiw') DESC )<=1)
  19 - filter("RN"=1)
  20 - filter(ROW_NUMBER() OVER ( PARTITION BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'
              fmiw') ORDER BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'fmiw') DESC )<=1)
  23 - filter("RN"=1)
  24 - filter(ROW_NUMBER() OVER ( PARTITION BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'
              fmiw') ORDER BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'fmiw') DESC )<=1)
  27 - filter("RN"=1)
  28 - filter(ROW_NUMBER() OVER ( PARTITION BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'
              fmiw') ORDER BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'fmiw') DESC )<=1)
  31 - filter("RN"=1)
  32 - filter(ROW_NUMBER() OVER ( PARTITION BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'
              fmiw') ORDER BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'fmiw') DESC )<=1)
  34 - access("PRESTACIONSERVICIO"."ID_SERVICIO"="SERVICIOS"."ID_SERVICIO")
  37 - filter("RN"=1)
  38 - filter(ROW_NUMBER() OVER ( PARTITION BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'
              fmiw') ORDER BY TRUNC(INTERNAL_FUNCTION("PRESTACIONSERVICIO"."FECHAHORA"),'fmiw') DESC )<=1)
  40 - access("PRESTACIONSERVICIO"."ID_SERVICIO"="SERVICIOS"."ID_SERVICIO")


PLAN_TABLE_OUTPUT
-------------------------------------------------------------------------------------------------------------------------------------------
Note
-----
   - dynamic statistics used: dynamic sampling (level=2)
   - 4 Sql Plan Directives used for this statement
```

Tiempos:

```
11:04:23,044 (AWT-EventQueue-0) DEBUG [DataNucleus.Datastore.Native] - select t1.week, t1.cantidad, t2.serviciomas, t3.serviciomenos, t4.ipsmas, t5.ipsmenos, t6.afiliado, t7.tipomenos, t8.tipomas from (     select week, (     select count(*) as cantidad     from afiliados     )-
count(*) as cantidad     from (         select TRUNC( prestacionServicio.fechaHora, 'IW' ) as week, prestacionservicio.numdoc         from prestacionServicio         Group by TRUNC( prestacionServicio.fechaHora, 'IW' ), numdoc     )     group by week ) t1 , (     select week,
serviciomas     from         select TRUNC( prestacionServicio.fechaHora,'IW' ) as week, id_servicio as serviciomas, count(*),             ROW_NUMBER() OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )             ORDER BY
TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn         from prestacionservicio         group by TRUNC( prestacionServicio.fechaHora,'IW' ), id_servicio         order by TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) desc         )     where rn = 1 ) t2 ,
(     select week, serviciomenos     from         (         select TRUNC( prestacionServicio.fechaHora,'IW' ) as week, id_servicio as serviciomenos, count(*),             ROW_NUMBER() OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )             ORDER BY
TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn         from prestacionservicio         group by TRUNC( prestacionServicio.fechaHora,'IW' ), id_servicio         order by TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) asc         )     where rn = 1 ) t3 ,
(     select week, ipsmas     from         (         select TRUNC( prestacionServicio.fechaHora,'IW' ) as week, id_ips as ipsmas, count(*),             ROW_NUMBER() OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )             ORDER BY
TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn         from prestacionservicio         group by TRUNC( prestacionServicio.fechaHora,'IW' ), id_ips         order by TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) desc         )     where rn = 1 ) t4 , (     select
week, ipsmenos     from         (         select TRUNC( prestacionServicio.fechaHora,'IW' ) as week, id_ips as ipsmenos, count(*),             ROW_NUMBER() OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )             ORDER BY
TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn         from prestacionservicio         group by TRUNC( prestacionServicio.fechaHora,'IW' ), id_ips         order by TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) asc         )     where rn = 1 ) t5 , (     select
week, afiliado     from         (         select TRUNC( prestacionServicio.fechaHora,'IW' ) as week, numdoc as afiliado, count(*),             ROW_NUMBER() OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )             ORDER BY
TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn         from prestacionservicio         group by TRUNC( prestacionServicio.fechaHora,'IW' ), numdoc         order by TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) asc         )     where rn = 1 ) t6 , (     select
week, tipomenos     from         (         select TRUNC( prestacionServicio.fechaHora,'IW' ) as week, servicios.tipo as tipomenos, count(*),             ROW_NUMBER() OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )             ORDER BY
TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn         from prestacionservicio inner join servicios         on prestacionservicio.id_servicio = servicios.id_servicio         group by TRUNC( prestacionServicio.fechaHora,'IW' ), servicios.tipo         order by
TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) desc         )         where rn = 1 ) t7 , (         select week, tipomas     from         (         select TRUNC( prestacionServicio.fechaHora,'IW' ) as week, servicios.tipo as tipomas, count(*),             ROW_NUMBER()
OVER(PARTITION BY TRUNC( prestacionServicio.fechaHora,'IW' )             ORDER BY  TRUNC( prestacionServicio.fechaHora,'IW' ) DESC) AS rn         from prestacionservicio inner join servicios         on prestacionservicio.id_servicio = servicios.id_servicio         group by
TRUNC( prestacionServicio.fechaHora,'IW' ), servicios.tipo         order by TRUNC( prestacionServicio.fechaHora,'IW' ), count(*) asc         )     where rn = 1 ) t8  where t1.week=t2.week and t2.week=t3.week and t3.week=t4.week and t4.week=t5.week  and t5.week=t6.week and
t6.week=t7.week and t7.week=t8.week  order by t1.week
11:04:23,935 (AWT-EventQueue-0) DEBUG [DataNucleus.Datastore.Retrieve] - SQL Execution Time = 891 ms
```

3.3.12 RFC12

Sentencias utilizadas:

```
-- afiliados que van cada mes.
select numdoc, count(*)
from (
    select afiliados.numdoc,
           TRUNC( prestacionServicio.fechaHora, 'MM' ) as meses
    from afiliados, prestacionservicio
    where afiliados.numdoc = prestacionservicio.numdoc
    group by afiliados.numdoc,
             TRUNC( prestacionServicio.fechaHora, 'MM' )
)
group by numdoc
having count(*)=24 ;

-- afiliados que siempre requieren servicios medicos especializados
select afiliados.numdoc, count(*) as total ,
       sum(case when servicios.tipo=7 then 1 else 0 end) as especial
from afiliados, prestacionservicio, servicios
where afiliados.numdoc = prestacionservicio.numdoc
and prestacionservicio.id_servicio = servicios.id_servicio
group by afiliados.numdoc
having count(*) = sum(case when servicios.tipo = 7 then 1 else 0 end)
;
```

Distribución de datos:

Se espera una distribución aleatoria de los servicios para cada afiliado por lo que la respuesta es demasiado grande para cada consulta, especialmente la última. Esto no es muy realista.

Planes de consulta de Oracle:

```
Plan hash value: 1486075473
-----------------------------------------------------------------------------------------------
| Id  | Operation            | Name               | Rows  | Bytes |TempSpc| Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT     |                    |   404 |  2020 |       |  1593   (1)| 00:00:01 |
|*  1 |  FILTER              |                    |       |       |       |            |          |
|   2 |   HASH GROUP BY      |                    |   404 |  2020 |       |  1593   (1)| 00:00:01 |
|   3 |    VIEW              | VM_NWVW_0          |   249K|  1215K|       |  1593   (1)| 00:00:01 |
|   4 |     HASH GROUP BY    |                    |   249K|  3161K|  6784K|  1593   (1)| 00:00:01 |
|   5 |      TABLE ACCESS FULL| PRESTACIONSERVICIO |   287K|  3655K|       |   343   (1)| 00:00:01 |
-----------------------------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter(COUNT(*)=24)
Note
-----
   - dynamic statistics used: dynamic sampling (level=2)
   - 1 Sql Plan Directive used for this statement
```

```
PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------------------
Plan hash value: 1827696054
-----------------------------------------------------------------------------------------------
| Id  | Operation            | Name                | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT     |                     |   404 |  5656 |   353   (3)| 00:00:01 |
|*  1 |  FILTER              |                     |       |       |            |          |
|   2 |   HASH GROUP BY      |                     |   404 |  5656 |   353   (3)| 00:00:01 |
|*  3 |    HASH JOIN         |                     |   287K|  3936K|   346   (1)| 00:00:01 |
|   4 |     TABLE ACCESS FULL| SERVICIOS           |    36 |   216 |     3   (0)| 00:00:01 |
|   5 |     TABLE ACCESS FULL| PRESTACIONSERVICIO  |   287K|  2249K|   343   (1)| 00:00:01 |

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------------------
Predicate Information (identified by operation id):
---------------------------------------------------
   1 - filter(COUNT(*)=SUM(CASE "SERVICIOS"."TIPO" WHEN 7 THEN 1 ELSE 0 END ))
   3 - access("PRESTACIONSERVICIO"."ID_SERVICIO"="SERVICIOS"."ID_SERVICIO")
```

Tiempos:

```
SQL Execution Time = 467 ms
```

4 Construcción de la aplicación

4.1 Diseño de datos

Para el diseño de datos para las tablas que hacen parte de la base de datos se usaron 6 roles de usuario, 15 IPS, 8 tipos de servicio, 36 servicios, 40.000 afiliados, 100 medicos, 45 rcepcionistas, 1 gerente, 1 administrador de dtaos y un organizador de campaña.

Tambien se poblaron las tablas de recetas, medicamentos e items recetas. En las tablas de ordenes se agregaron un total de 360.000 ordenes, distribuidas aleatoriamente a lo largo de los años 2018 y 2019, en los que los servicios asignados tambien eran asignados aleatoriamente. De estas ordenes solo el 90% tenian una reserva y el 80% presentaban una prestacion del servicio por lo que las tablas de reservas y prestacion tienen correspondientemete, 324.000 y 288.000. Tambien se agregan 3 campañas a lo largo de estos años.

Para lograr obtener este volumen de datos se escribio un programa en python que escribiera en un archivo de texto las sentencias sql necesarias para poblar las tablas, debido a la gran cantidad de tiempo que lleva a cabo la poblacion de las tablas mediante este proceso se uso sqlLoader con el que con el uso de un excel (generado con python), se cargan datos a las tablas de manera mas eficiente.

4.2 Análisis de optimización

La gran diferencia entre la ejecución de consultas delegada al manejador de bases de datos a diferencia de traer los datos a memoria principal es el tipo de estructuras de datos que se utilizan, así como el volumen de datos y verdaderamente la posibilidad de hacerlo. Cuando se

tienen demasiados datos, estos puede que no quepan en memoria principal, por lo que se debe comenzar a utilizar la memoria secundaria. Al trabajar en memoria secundaria las estructuras cambian y la complejidad temporal deja de depender de las instrucciones de control únicamente. Aquí la complejidad comienza a depender de las lecturas y escrituras en el disco, las cuales van a tener mucho más peso que las instrucciones de control.

Si se quisiera implementar este tipo de funciones con memoria secundaria localmente sería difícil alcanzar la eficiencia de un SMBD como Oracle puesto que no solo tiene estructuras de datos especialmente diseñadas para la optimización de la memoria secundaria, también cuenta con mecanismos de optimización como es el optimizador de consultas, el cual se apoya a su vez en estructuras de índices. Este es el verdadero valor agregado de estas aplicaciones que permiten que operaciones complejas como un join, que en teoría es un producto cartesiano, es decir una multiplicación de todas las tuplas con todas, sea mucho más eficiente. Permite además guardar vistas y crear índices personalizados.

Claro está que el optimizador de Oracle no es perfecto y a veces un plan de ejecución pueda no ser la decisión que uno tomaría. De hecho el optimizador en sí a veces puede tomar más tiempo en crear el plan de ejecución que en ejecutarlo. No obstante, es una herramienta vital para el manejo de operaciones con grandes números de datos. Con tan solo EPSAndes que tiene un millón de datos se puede ver la diferencia en el manejo de los datos frente a los programas creados en la clase de estructuras de datos a pesar de tener un volumen de datos muy similar.

5 Resultados

5.1 Resultados logrados

Se logró llevar a cabo los requerimientos funcionales, poblar las tablas con más de un millón de datos en total y asegurar la eficiencia de las consultas.

5.2 Resultados no logrados

Ninguno.